

## Стилизация элементов через атрибут style в JavaScript

Давайте теперь научимся добавлять CSS стили элементам. Это делается путем изменения атрибута `style`. К примеру, чтобы поменять цвет, нужно построить следующую цепочку: `elem.style.color`, и присвоить ей нужное значение цвета.

Посмотрим на примере. Пусть у нас есть вот такой элемент:

```
1 | <p id="elem">text</p>
```

Сделаем этот элемент красного цвета:

```
1 | let elem = document.querySelector('#elem');  
2 | elem.style.color = 'red';
```



№ 1

©jsPmStSA

Дан див и кнопка. По клику на кнопку добавьте диву ширину, высоту и границу.

## Стилизация свойств с дефисом в JavaScript

Свойства, которые записываются через дефис, например `font-size`, преобразуются в `camelCase`. То есть `font-size` станет `fontSize`:

```
1 | elem.style.fontSize = '20px';
```

№ 1

©jsPmStHP

Дан див с текстом и кнопка. По клику на кнопку установите диву размер шрифта в 20px, а также верхнюю границу и фон.

## Исключение при стилизации элементов в JavaScript

Свойство `float` является исключением, так как оно является специальным в JavaScript. Для него следует писать `cssFloat`:

```
1 | elem.style.cssFloat = 'right';
```



№ 1

©jsPmStEx

Дан список `ul` и кнопка. По клику на кнопку добавьте тегам `li` свойство `float` в значении `left`.

## Стилизация с помощью CSS классов на JavaScript

В предыдущем уроке мы научились менять CSS стили элементов через изменение атрибута `style`. Чаще всего это не очень хорошая идея. Дело в том, что, если раскидать CSS стили по JavaScript коду, в дальнейшем будет проблематично изменить дизайн сайта, так как придется ковырять JavaScript код в поисках зашитых туда стилей.

Более удобно для дальнейшей поддержки будет добавлять или убирать элементу CSS классы, тем самым стилизуя их нужным образом.

Давайте посмотрим на примере. Пусть у нас есть несколько абзацев:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Давайте сделаем так, чтобы по клику на любой абзац, этот абзац красился в какой-нибудь цвет, например, в зеленый. Для этого в CSS файле сделаем специальный класс для окрашивания абзацев:

```
1 .colored {
2   color: green;
3 }
```

Преимущество использование класса в том, что теперь легко можно будет поменять желаемый цвет - для этого нужно будет просто внести изменение в CSS файл, не ковыряя JavaScript код. Особенно удобно это будет в том случае, если JavaScript код пишете вы, а в дальнейшем стилизовать его будет кто-то другой (возможно менее квалифицированный человек, умеющий работать только с CSS).

Итак, теперь, после введения класса, по клику на любой абзац можно изменить его цвет, просто добавив ему наш класс:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     this.classList.add('colored'); // добавляем абзацу класс
6   });
7 }
```

## № 1

⊗jsPmStCl

Объясните, почему я выбрал для названия класса слово `colored`, а не слово `green`, явно указывающее на желаемый нами цвет.

## № 2

⊗jsPmStCl

Дан абзац. Даны кнопки 'перечеркнуть', 'сделать жирным', 'сделать красным'. Пусть по нажатию на каждую кнопку заданное действие происходит с абзацем (становится красным, например).

## Преимущество стилизации с помощью CSS классов в JavaScript

Использование классов вместо изменения стилей напрямую имеет еще одно преимущество. Легким движением руки можно сделать так, что стили элементов будут чередоваться.

Например, можно сделать так, что при первом клике на абзац он будет красится в определенный цвет, а при повторном клике - возвращать себе исходный цвет. Для этого нужно просто метод `add` поменять на метод `toggle`:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     this.classList.toggle('colored');
6   });
7 }
```

Модифицируйте предыдущую задачу так, чтобы повторное нажатие на кнопку отменяло действие этой кнопки.

## Применение стилизации в JavaScript

Давайте сделаем кнопку, по нажатию на которую элемент будет то показываться, то скрываться. Пусть по умолчанию элемент скрыт (это реализуем с помощью `display: none`), а покажется он с помощью добавления класса `active`. Этот класс будем то добавлять, то убирать с помощью `classList.toggle`:

```
1 <button id="button">click me</button>
2 <div id="elem"></div>

1 #elem {
2   display: none;
3   width: 200px;
4   height: 200px;
5   border: 1px solid green;
6 }
7 #elem.active {
8   display: block;
9 }

1 let button = document.querySelector('#button');
2 let elem = document.querySelector('#elem');
3
4 button.addEventListener('click', function() {
5   elem.classList.toggle('active');
6 });
```

